

JavaScript Notes – Loops, Prime Numbers & Array Functions

1. Loops in JavaScript

Loops are used to execute a block of code repeatedly.

2. for Loop

Syntax

```
for(initialization; condition; increment/decrement){  
    // code  
}
```

Example

```
for(let i = 1; i <= 5; i++){  
    console.log(i);  
}
```

Output

```
1  
2  
3  
4  
5
```

3. Even Numbers

Definition

Even numbers are numbers divisible by 2.

Logic

```
number % 2 == 0
```

If remainder is 0, then number is even.

Program

```
for (let i = 1; i <= 20; i++) {  
    if (i % 2 == 0) {  
        console.log(i);  
    }  
}
```

```
}
```

Output

```
2
4
6
8
10
12
14
16
18
20
```

4. Odd Numbers

Definition

Odd numbers are NOT divisible by 2.

Logic

```
number % 2 !== 0
```

Program

```
for (let i = 1; i <= 20; i++) {
    if (i % 2 !== 0) {
        console.log(i);
    }
}
```

Output

```
1
3
5
7
9
11
13
15
17
19
```

5. Prime Numbers

Definition

A prime number:

- Is greater than 1
 - Divisible only by:
 - 1
 - itself
-

Examples

Prime Numbers:

2, 3, 5, 7, 11

Non Prime Numbers:

4, 6, 8, 9

6. Prime Number Logic

Suppose:

userInput = 7

We check:

```
7 % 2
7 % 3
7 % 4
7 % 5
7 % 6
```

If any remainder becomes 0,
then the number is NOT prime.

7. Prime Number Program

```
let userInput = 6;
let count = 0;

for (let i = 2; i < userInput; i++) {

    let remainder = userInput % i;

    if (remainder == 0) {
        count++;
    }

}

if (count == 0) {
    console.log("Prime Number");
}
else {
    console.log("Not a Prime Number");
}
```

8. Dry Run Example

userInput = 6

i 6 % i Divisible?

2 0 Yes

3 0 Yes

4 2 No

5 1 No

Since divisible by other numbers:

Not Prime

9. while Loop

Syntax

```
while(condition){  
    // code  
}
```

Example

```
let i = 1;  
  
while(i <= 5){  
    console.log(i);  
    i++;  
}
```

Output

```
1  
2  
3  
4  
5
```

10. Infinite Loop

```
while(true){  
  
}
```

This loop never stops.

⚠ Never use infinite loops accidentally because browser may freeze.

11. Nested Loops

Definition

A loop inside another loop.

Example: Multiplication Tables

```
let table = 5;
let count = 10;

for(let i = 1; i <= table; i++){
    for(let j = 1; j <= count; j++){
        console.log(`${i} * ${j} = ${i*j}`);
    }
}
```

12. How Nested Loop Works

Outer loop:

i = 1

Inner loop runs completely:

1*1
1*2
1*3
...
1*10

Then outer loop changes:

i = 2

Again inner loop runs completely.

13. Arrays in JavaScript

Definition

Arrays store multiple values in a single variable.

Example

```
let array = [12, 12.12, "string", true, false, 12];
```

Array Indexes

Index	Value
-------	-------

0	12
---	----

Index	Value
1	12.12
2	"string"
3	true
4	false
5	12

14. push()

Purpose

Adds element at the END of array.

Example

```
array.push("new element");
```

Before

```
[12, 12.12, "string"]
```

After

```
[12, 12.12, "string", "new element"]
```

15. pop()

Purpose

Removes last element.

Example

```
array.pop();
```

16. unshift()

Purpose

Adds element at START.

Example

```
array.unshift("new element at start");
```

17. shift()

Purpose

Removes first element.

Example

```
array.shift();
```

18. indexOf()

Purpose

Returns first index of element.

Example

```
console.log(array.indexOf(12));
```

Output

0

19. lastIndexOf()

Purpose

Returns last occurrence index.

Example

```
console.log(array.lastIndexOf(12));
```

Output

5

20. includes()

Purpose

Checks whether element exists.

Returns:

- true
 - false
-

Example

```
console.log(array.includes(22));
```

Output

false

21. slice()

Purpose

Extracts part of array.

Original array remains unchanged.

Syntax

```
array.slice(start, end)
```

- Start included
 - End excluded
-

Example

```
console.log(array.slice(1,4));
```

Output

[12.12, "string", true]

22. splice()

Purpose

Used to:

- Remove elements
 - Add elements
 - Replace elements
-

Syntax

```
array.splice(startIndex, deleteCount, newElement)
```

Example

```
array.splice(4, 2, "new element");
```

Explanation

Part	Meaning
4	Start from index 4
2	Remove 2 elements
"new element"	Add this element

23. Before and After splice

Before

```
[12, 12.12, "string", true, false, 12]
```

After

```
[12, 12.12, "string", true, "new element"]
```

24. Difference Between slice() and splice()

slice()	splice()
Does NOT modify original array	Modifies original array
Used for copying	Used for add/remove
Returns new array	Changes same array

25. Complete Combined Program

```
// EVEN NUMBERS
for (let i = 1; i <= 20; i++) {

    if (i % 2 == 0) {
        console.log(i);
    }

}
```

```
// ODD NUMBERS
for (let i = 1; i <= 20; i++) {

    if (i % 2 !== 0) {
        console.log(i);
    }

}
```

```

}

// PRIME NUMBER
let userInput = 6;
let count = 0;

for (let i = 2; i < userInput; i++) {

    if (userInput % i == 0) {
        count++;
    }

}

if (count == 0) {
    console.log("Prime Number");
}
else {
    console.log("Not a Prime Number");
}

// ARRAY FUNCTIONS
let array = [12, 12.12, "string", true, false, 12];

array.push("new element");
array.pop();

array.unshift("new element at start");
array.shift();

console.log(array.indexOf(12));

console.log(array.lastIndexOf(12));

console.log(array.includes(22));

console.log(array.slice(1, 4));

array.splice(4, 2, "new element");

console.log(array);

```

26. Important Interview Questions

Q1. Difference between push() and unshift()

push()	unshift()
Adds at end	Adds at start

Q2. Difference between pop() and shift()

pop()	shift()
Removes last element	Removes first element

Q3. Difference between slice() and splice()

slice()	splice()
---------	----------

slice()

splice()

Does not change original array Changes original array

27. Practice Questions

Loops

1. Print numbers from 1 to 100
 2. Print all even numbers
 3. Print all odd numbers
 4. Print multiplication table of 7
-

Prime Numbers

5. Find all prime numbers from 1 to 50
 6. Check if a number is prime
-

Arrays

7. Add element using `push()`
8. Remove element using `pop()`
9. Find index using `indexOf()`
10. Replace element using `splice()`